

Worst-Case Time Complexity Calculation using LLM

Manish Kumar (SC24M147)
MA642 – Data Modelling Lab II
Indian Institute of Space Science and Technology

Abstract—Accurately estimating the time complexity of algorithms is essential for optimizing code performance and making informed computational trade-offs. Traditional methods for complexity analysis often rely on manual inspection or static code analysis, which can be time-consuming and prone to human error. In this project, we fine-tune DeepSeek Coder 1.3B, a large language model, to predict the time complexity of Python code automatically.

We construct a specialized dataset comprising Python code snippets annotated with their computational complexity labels (e.g., $O(n)$, $O(\log n)$, $O(n^2)$). To enhance feature representation, we extract key structural metrics such as loop count, maximum loop depth, and recursion frequency using Abstract Syntax Tree (AST) analysis. The fine-tuning process employs parameter-efficient techniques such as Low-Rank Adaptation (LoRA) and mixed-precision training (FP16) to optimize memory usage while maintaining model performance.

The model is trained on high-memory GPUs (12–16GB VRAM), leveraging Hugging Face Transformers and PyTorch to ensure scalability and efficiency. By automating complexity analysis, this project aims to assist developers in assessing algorithm efficiency, improving code optimization workflows, and reducing the reliance on manual complexity evaluation.

I. INTRODUCTION

THE idea is to analyze Python code snippets to determine their worst-case time complexity (e.g., $O(n)$, $O(\log n)$) using a fine-tuned LLM. One key challenge was the limited GPU memory (15 GB), requiring strategies for efficient resource utilization. The work falls under deterministic classification problems, where each input Python code snippet is mapped to a single output complexity class.

II. DATASET

A curated dataset of Python code snippets labeled with their worst-case time complexities was used. Structural metrics were auto-extracted using the Python AST module, including:

- Number of loops
- Maximum nesting depth
- Number of recursive calls

This helped in enhancing the prompt engineering and model input formatting.

Code	Label
Print("Dataset")	$O(1)$
for i in range(n): print("Dataset")	$O(n)$
for i in range(n): for j in range(n) print("Dataset")	$O(n^2)$
for i in range(n): for j in range(m) print("Dataset1")	$O(n*m)$
⋮	⋮
⋮	⋮
⋮	⋮

Fig. 1. Dataset example with metrics

Label	Sample Size
$O(1)$	100
$O(n)$	100
$O(n^2)$	100
$O(n*m)$	100
$O(\log n)$	100
$O(n * \log n)$	100
$O(n^3)$	100
$O(2^n)$	100
$O(n^m)$	100
$O(n!)$	100
Total	1000

Fig. 2. Dataset size and distribution

III. MODEL SELECTION

The base model used is [DeepSeek Coder-1.3B Instruct](#), a transformer model with 1.3 billion parameters trained on 2T tokens (87% code, 13% natural language). This model supports multiple programming languages and is optimized for code understanding tasks.

System Configuration:

- Platform: Google Colab
- GPU: T4 / P100 with 15 GB VRAM
- CPU RAM: 12.7 GB

IV. FINE-TUNING PIPELINE

A. Environment Setup

Dependencies: transformers, datasets, accelerate, peft, bitsandbytes,

huggingface_hub, etc.

Versions:

- PyTorch: 2.1
- Transformers: 4.34
- PEFT: 0.3
- BitsAndBytes: 0.41

B. Model Quantization

Used 4-bit quantization with `BitsAndBytesConfig`, reducing model size from ~ 5 GB to ~ 1.3 GB and GPU memory usage to ~ 2.6 GB. This enabled efficient training even on hardware with limited VRAM.

C. LoRA Adapter Injection

Low-Rank Adaptation (LoRA) was used to inject trainable adapters into attention layers:

- Rank $r = 16$, $\alpha = 32$, Dropout = 0.05
- Targeted modules: `q_proj`, `v_proj`
- Trainable Params: ~ 0.2 M (0.015%)
- Additional VRAM: ~ 300 MB

D. Data Formatting and Tokenization

Prompt Template:

```
### Instruction: what is the worst-case
time complexity of this Python code
### <code>
### <auto-metrics>
### Response: O(n)
```

Input sequences were truncated/padded to 512 tokens and formatted as PyTorch tensors.

E. Training Configuration

Training settings used:

- Epochs: 5, Batch Size: 2
- Gradient Accumulation: 4
- Learning Rate: $2e-4$
- Optimizer: `paged_adamw_8bit`, FP16 enabled
- Logging every 10 steps

Training Notebook: [Google Colab Link](#)

V. KEY CHALLENGES AND SOLUTIONS

- **Limited GPU Memory:** Only 15 GB of VRAM was available, which required optimized techniques like quantization and adapter-based fine-tuning (LoRA).
- **Feature Extraction:** Auto-metrics like loop count, recursion depth, and nesting were extracted using Python AST, allowing a structured understanding of the code.
- **Parameter Efficiency:** LoRA reduced the need to update full model weights, enabling the training of just 0.015% of parameters, making it feasible on restricted resources.

Challenge	Solution
1.3 B parameters won't fit 15 GB at FP32	4-bit quantization (NF4)
Full-model fine-tuning too heavy	LoRA adapters (~ 0.2 M trainable)
Large activation & optimizer states	8-bit Adam + CPU offload

Fig. 3. Challenges and solutions overview

VI. INFERENCE AND EVALUATION

A. Inference

Inference used deterministic decoding with `temperature = 0.0`. The same prompt structure from training was reused.

Inference Notebook: [Google Colab Link](#)

B. Evaluation

Validation Set: 100 code samples (10 per complexity class)
Accuracy: 80.81%

Evaluation metrics confirmed that the model learned the association between code patterns and complexity labels effectively, although nested versus sequential loop distinctions posed occasional difficulty.

VII. DEPLOYMENT

The fine-tuned model was uploaded to Hugging Face for public access.

Repo: [malviya7665/deepseek-complexity-1.3b](#)

Artifacts:

- `adapter_model.safetensors`
- `adapter_config.json`
- `tokenizer.json` and `tokenizer_config.json`

Note: Only LoRA adapters and tokenizer are stored. Base model must be loaded during inference.

VIII. CONCLUSION

This project successfully demonstrates that even with limited resources, fine-tuning large LLMs for deterministic code analysis tasks is feasible. The model achieves over 80% accuracy in time complexity classification, making it a viable tool for static code analysis and development automation. The integration with Hugging Face allows scalable deployment.